

© Brendan Choi 2021

B. Choi, *Introduction to Python Network Automation*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-6806-3_19

19. Python Network Automation Labs: Combining and Completing the Cisco IOS Upgrade Application

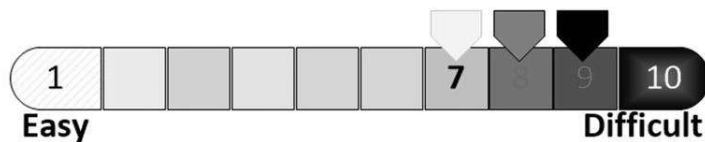
Brendan Choi¹

(1) Sydney, NSW, Australia

In this final chapter, you will put together the ten Python stand-alone applications we developed in Chapter 18. The application will be a fully functional Cisco IOS/IOS-XE upgrade application consisting of pre-check, IOS upload, pre-reload check, pre-reload configuration backup, reload, and post-upgrade verification check. We have been upgrading IOS on Cisco network devices manually for years, but now you can develop and combine various Python tools to let your Python code do to work for you.

Once you have reached the end of this chapter, you will have learned how to put together smaller applications to make a sophisticated and production-ready application. Python allows programmers the freedom to write almost any code they like, and the only limitation is the programmer's imagination. On the other hand, configuration and orchestration tools such as Ansible and Puppet provide nonprogramming engineers with idempotent features to configure and manage network devices with a lot of flexibility. However, you still have to work within the framework set by production. After completing this chapter, and hence this book, you will have reached the first milestone in your Python network programming journey.

****Difficulty level will be relative to reader's previous experience
with Cisco Networking Technology.***



In the previous chapter, you developed ten applications related to the Cisco IOS upgrade. Now it is time to combine those tools and make a fully functioning Cisco IOS XE upgrade tool. As you work on more Python projects, you will always find something new that you have not encountered before. One of the best ways to learn is through project work with lots of unknowns and challenges. However, if you are starting your Python network automation journey, then try to find a smaller task to focus your efforts and document what tasks you can. Start to automate more straightforward tasks and then spend the time researching and studying books, taking online courses, searching the information on

Google, or seeking advice from others who have already done something similar.

I briefly introduced you to object-oriented programming (OOP) at the beginning of the previous chapter. We will not use it here in our final IOS XE upgrade application as it adds more complexity. Even though OOP is a must-understand concept, we can still write code without creating custom OOP classes. In Python, almost everything can be considered an object, and even if you are not aware of OOP, you are still using it. Please note that I have purposefully removed the OOP portion of the final script, and it will not be part of our final Python application.

You have made it to the final exercise, which is the book's real climax. I want to congratulate you on a mammoth effort for following along with all the chapters of the book. A few years back, when I began my first journey in network automation with Python, it was still considered a relatively new technology trend and felt like a distant dream to automate tasks such as a Cisco IOS upgrade on enterprise routers and switches. Network automation has recently gained momentum, and now network automation and Python coding are on every IT/network team's mind. So without further ado, let's get on with completing the final application: a working IOS upgrade tool.



Before starting your final tasks in this section, please review Figure 17-14 and Figure 17-15 in Chapter 17. You may need a quick refresher to gauge what we are about to achieve in this final lab. The final script's workflow will be almost identical to the automation tasks described in Figure 17-15.

Creating a Single Python File

For simplicity, we will create only a single Python file (`upgrade_crs1000v.py`). In a real-world production environment, the best practice is to create and save tools into smaller files (modules) to reduce the number of lines written to the main script. However, here we are still exploring and learning, so it is more important to see the whole picture of how each application works. After you have finished the book, you can move some parts of the code into separate modules, which was discussed in the previous chapter. As discussed earlier, we will make this application an interactive tool, getting the user credentials and user inputs while the script runs. The device information will be written to a `.csv` file and read from the file using the `pandas` module as `df` (dataframe); this is a more convenient way to read a large text-based data than to make the user enter the data.

Task

You will be continuing the tasks from the `/home/pynetauto/my_tools/` directory; this directory contains all the tasks from the previous chapter. You will be continuing the tasks from the `/home/pynetauto/my_tools/` directory; this directory contains all the tasks from the previous chapter. You will be continuing the tasks from the `/home/pynetauto/my_tools/` directory; this directory contains all the tasks from the previous chapter. You will be continuing the tasks from the `/home/pynetauto/my_tools/` directory; this directory contains all the tasks from the previous chapter.

1 `upgrade_crs1000v.py` file inside this directory now.

```
pynetauto@ubuntu20s1:~$ pwd
```

```
/home/pynetauto
```

```
pynetauto@ubuntu20s1:~$ cd my_tools
```

Task

```
pynetauto@ubuntu20s1:~/my_tools$ touch upgrade_crs1000v.py
```

```
pynetauto@ubuntu20s1:~/my_tools$ ls
```

```
upgrade_crs1000v.py
```

We will start filling in the final `upgrade_crs1000v.py` script soon; leave it there as a placeholder for the time

2

Let's quickly discuss which files will be referenced for our application development. If you followed Chapter 1, you should have created the following directory names. The names of the files you will reference are listed next. If you placed the files to the `/home/pynetauto/my_tools/` directory for reference, but for the completed IOS applications to the `/home/pynetauto/my_tools/new_ios/` directory and also a new IOS file in the `/home/pynetauto/my_tools/new_ios/` directory. If you have completed uploading the new IOS file (`csr1000v-universalk9.16.09.06.SPA.bin`) yet, refer to the IOS uploading section.

```
pynetauto@ubuntu20s1:~/my_tools$ ls -lh
```

```
total 52K
```

```
# Main script and files
```

```
-rw-rw-r-- 1 pynetauto pynetauto    0 Jan 15 22:03 upgrade_crs1000v.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto  271 Jan 15 23:13 devices_info.csv
```

```
# Python application scripts referenced:
```

```
-rw-rw-r-- 1 pynetauto pynetauto  955 Jan 15 23:14 check_flash2.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto 1.5K Jan 15 23:13 get_cred2.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto 1.8K Jan 15 23:18 make_backup1.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto 1.3K Jan 15 23:14 md5_validate3.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto 2.4K Jan 15 23:18 md5_verify2.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto  681 Jan 15 23:14 ping_tool6.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto 1009 Jan 15 23:14 ping_tool6_tools.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto 6.1K Jan 15 23:18 post_check2.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto 1.3K Jan 15 23:13 read_info8.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto 3.9K Jan 15 23:18 reload_yesno2.py
```

```
-rw-rw-r-- 1 pynetauto pynetauto 3.2K Jan 15 23:14 upload_ios1.py
```

Task

```
# new IOS .bin file location
```

```
pynetauto@ubuntu20s1:~/my_tools$ ls /home/pynetauto/my_tools/new_ios
```

```
csr1000v-universalk9.16.09.06.SPA.bin
```

3 If you do not have the `devices_info.csv` file from the previous chapter, you can create it now. You can create it with any name (but no spaces). After creating your file, it should look the same as the following:

```
pynetauto@ubuntu20s1:~/my_tools$ nano devices_info.csv
```

```
devices_info.csv
```

```
devicename,device,devicetype,host,newios,newiosmd5
```

```
csr1000v-1,RT,cisco_xe,192.168.183.111,csr1000v-universalk9.16.09.06.SPA.bin,77878ae6db8e1
```

```
csr1000v-2,RT,cisco_xe,192.168.183.222,csr1000v-universalk9.16.09.06.SPA.bin,77878ae6db8e1
```

4 Now that the standard procedures are out of the way, let's begin the final application development. You will be combining all the pieces together, it will work as an application that can be directly used in most production environments.

First, you will add all the Python modules that will be used in our application and then add and modify the code to end.

```
pynetauto@ubuntu20s1:~/my_tools$ nano upgrade_crs1000v.py
```

Add all the modules used, as shown here:

```
upgrade_crs1000v.py
```

```
import socket # For socket networking
```

```
import os # For Python Server OS
```

```
import time # Python time module
```

```
import pandas as pd # Pandas to read data into a data frame
```

```
import re # Python regular expression module
```

```
from getpass import getpass # For password collection
```

```
import os.path # For Python Server OS directory
```

```
import hashlib # For MD5 checks
```

```
from netmiko import ConnectHandler, SCPConn # For netmiko's SSH connection and SCP file transfer
```


Task

```
from netmiko.ssh_exception import NetMikoTimeoutException # To accept netmiko timeout exc
```

```
import difflib # For analyzing two files and differences
```

```
t = time.mktime(time.localtime()) # Timer start to measure script running time
```

In the next step, you will append and modify sections of code from each tool and slowly build up the main a

- 5 A. Use `get_cred2.py (tool2_login)` and turn it into a function. This script will interactively get the network using a regular expression to control the user's input for the username and password. For `p1`, the expected characters are from 8 to 50 characters long, starting with a letter and finishing with any character. R

```
"""
```

```
Step 1. From get_cred2.py (tool2_login) application :
```

```
Get network administrator ID and password, also prompt for enable secret and give the user
```

```
p1: Between 5-30 characters long, starting with an alphabet & finishes with an alphabet or
```

```
p2: Between 8 to 50 characters long, starting with an alphabet & finishes with any charact
```

```
"""
```

```
def get_secret(p2):
```

```
    global secret
```

```
    resp = input("Is secret same as password? (y/n) : ")
```

```
    print("-"*80)
```

```
    resp = resp.lower()
```

```
    if resp == "yes" or resp == "y":
```

```
        secret = pwd
```

```
    elif resp == "no" or resp == "n":
```

```
        secret = None
```

```
    while not secret:
```

```
        secret = getpass("Enter the secret : ")
```

```
    while not p2.match(secret):
```

```

# Task

secret = getpass(r"*Enter the secret : ")

secret_verify = getpass("Confirm the secret : ")

if secret != secret_verify:

    print("!!! the secret does not match. Please try again.")

    print("-"*80)

    secret = None

else:

    get_secret(p2)

def get_credentials():

    p1 = re.compile(r'^[a-zA-Z][a-zA-Z0-9_-]{3,28}[a-zA-Z0-9]$')

    p2 = re.compile(r'^[a-zA-Z]{7,49}')

    global uid

    uid = input("Enter Network Admin ID : ")

    while not p1.match(uid):

        uid = input(r"*Enter Network Admin ID : ")

    global pwd

    pwd = None

    while not pwd:

        pwd = getpass("Enter Network Admin PWD : ")

        while not p2.match(pwd):

            pwd = getpass(r"*Enter Network Admin PWD : ")

        pwd_verify = getpass("Confirm Network Admin PWD : ")

        if pwd != pwd_verify:

            print("!!! Network Passwords do not match. Please try again.")

```

Task

```
print("-"*80)
```

```
pwd = None
```

```
# Trigger get_secret function to run
```

```
get_secret(p2)
```

```
return uid, pwd, secret
```

```
# Trigger get_Credential function to run
```

```
get_credentials()
```

Now you have completed the above task A, continue with the task B.

6 B. Copy and paste `read_info8.py (tool3_read_csv)`; this is the script that reads the `devices_info.csv` file portability and readability. Ensure that you pass the three arguments received from task A, so that the script can read the `uid`, `pwd`, and `secret`.

```
"""
```

```
Step 2. From read_info8.py (tool3_read_csv) application:
```

```
Read the content of the devices_info.csv file and convert the values into two lists. device
friendly dictionary items for SSH connection
```

```
"""
```

```
def read_info(uid, pwd, secret):
```

```
df = pd.read_csv(r'./devices_info.csv') # ensure the correct file location
```

```
number_of_rows = len(df.index)
```

```
# Read the values and save as a list, read column as df and save it as a list
```

```
devicename = list(df['devicename'])
```

```
device = list(df['device'])
```

```
devicetype = list(df['devicetype'])
```

```
ip = list(df['host'])
```

```
newios = list(df['newios'])
```

```
newiosmd5 = list(df['newiosmd5'])
```

```
# Task
```

```
# Append the items and convert to a list, device_list
```

```
global device_list # For md5_validate3.py
```

```
device_list = []
```

```
for index, rows in df.iterrows():
```

```
    device_append = [rows.devicename, rows.device, \
```

```
                    rows.devicetype, rows.host, rows.newios, rows.newiosmd5]
```

```
    device_list.append(device_append)
```

```
# Using device_list, create a netmiko friendly list device_list_netmiko
```

```
global device_list_netmiko
```

```
device_list_netmiko = []
```

```
i = 0
```

```
for x in device_list:
```

```
    if len(x) != 0: # As long as number of items in device_list is not 0 (empty)
```

```
        i += 1
```

```
        name = f'device{str(i)}' # Each for loop, the name is updated to device1, device2, de
```

```
        devicetype, host = x[2], x[3]
```

```
        device = {
```

```
            'device_type': devicetype,
```

```
            'host': host,
```

```
            'username': uid,
```

```
            'password': pwd,
```

```
            'secret': secret,
```

```
        }
```

```
        device_list_netmiko.append(device)
```

Task

```
# Trigger read_info function to run
```

```
read_info(uid, pwd, secret)
```

7 C. This module is almost straight out of `ping_tool6.py` and `ping_tool6_tools.py` (`tool1_ping`). Copy the `ping` command and socket networking to check for an open socket 22 status. If an IP address is unreachable This script should produce three separate files containing reachability status. If port 22 is open, then the de the device is unreachable, then the IP address will be written to `f3`.

```
"""
```

```
Step 3. From ping_tool6.py and ping_tool6_tools.py (tool1_ping) application:
```

```
Perform connectivity tests, first using ICMP ping, Second, check port 22. Also, separate t
result of the reachability tests
```

```
"""
```

```
def test_connectivity(device_list_netmiko):
```

```
    f1 = open('reachable_ips_ssh.txt', 'w+')
```

```
    f2 = open('reachable_ips_telnet.txt', 'w+')
```

```
    f3 = open('unreachable_ips.txt', 'w+')
```

```
    for device in device_list_netmiko:
```

```
        ip = device['host'].strip()
```

```
        print(ip)
```

```
        resp = os.system('ping -c 4 ' + ip)
```

```
        if resp == 0:
```

```
            for port in range (22, 23):
```

```
                destination = (ip, port)
```

```
                try:
```

```
                    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
```

```
                        s.settimeout(3)
```

```
                        connection = s.connect(destination)
```

```
# Task
```

```
print(f"{ip} {port} open")
```

```
print("-"*80)
```

```
f1.write(f"{ip}\n")
```

```
except:
```

```
print(f"{ip} {port} closed")
```

```
f3.write(f"{ip} {port} closed\n")
```

```
for port in range (23, 24):
```

```
destination = (ip, port)
```

```
try:
```

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
```

```
s.settimeout(3)
```

```
connection = s.connect(destination)
```

```
print(f"{ip} {port} open")
```

```
print("-"*80)
```

```
f2.write(f"{ip}\n")
```

```
except:
```

```
print(f"{ip} {port} closed")
```

```
print("-"*80)
```

```
f3.write(f"{ip} {port} closed\n")
```

```
else:
```

```
print(f"{ip} unreachable")
```

```
print("-"*80)
```

```
f3.write(f"{ip} unreachable\n")
```

```
f1.close()
```

```
# Task
```

```
f2.close()
```

```
f3.close()
```

```
# Trigger test_connectivity function to run
```

```
test_connectivity(device_list_netmiko)
```

8 D. This part is from md5_validate3.py (tool4_md5_linux) , and as explained earlier, this will validate the new IOS is in the correct directory. use Python's hashlib to test the IOS's MD5 value in a directory. Testing the MD5 values rigorously through new IOS is in the correct directory.

```
"""
```

```
Step 4. From md5_validate3.py (tool4_md5_linux) application :
```

```
Validate the MD5 value of the new IOS file on the Python Server against the value you have
```

```
"""
```

```
def validate_md5(device_list):
```

```
    for x in device_list:
```

```
        print(x[3], x[0], x[1], x[2])
```

```
        newios = x[4]
```

```
        newiosmd5 = str(x[5].lower()).strip()
```

```
        print(newiosmd5)
```

```
        newiosmd5hash = hashlib.md5()
```

```
        file = open(f'/home/pynetauto/my_tools/new_ios/{newios}', 'rb') # New IOS directory he
```

```
        content = file.read()
```

```
        newiosmd5hash.update(content)
```

```
        newiosmd5server = newiosmd5hash.hexdigest()
```

```
        print(newiosmd5server.strip())
```

```
    global newiossize
```

```
    newiossize = round(os.path.getsize(f'/home/pynetauto/my_tools/new_ios/{newios}')/10000
```

```
# Task
```

```
print(newiossize, "MB")
```

```
if newiosmd5server == newiosmd5:
```

```
print("MD5 values matched!")
```

```
print("-"*80)
```

```
else:
```

```
print("Mismatched MD5 values. Exit")
```

```
print("-"*80)
```

```
exit()
```

```
return newiossize
```

```
# Trigger validate_md5 function to run
```

```
validate_md5(device_list)
```

9

E. In this task, the script is almost a carbon copy of `check_flash2.py` (`tool5_fsize_cisco`). As you create a `device_list_netmiko` list for the SSH connection and in the variable `newiossize` to confirm that the route half times the new IOS file size, the script will exit. On newer Cisco devices, the flash free size is not much of the old IOS from the flash to make room. You could write something funky here and automatically locate the

```
"""
```

```
Step 5. From check_flash2.py (tool5_fsize_cisco) application :
```

```
Checks available flash size on the router
```

```
"""
```

```
def check_flash(device_list_netmiko, newiossize):
```

```
for device in device_list_netmiko:
```

```
ip = str(device['host'])
```

```
net_connect = ConnectHandler(**device)
```

```
net_connect.send_command("terminal length 0")
```

```
showdir = net_connect.send_command("dir")
```

```
#showflash = net_connect.send_command("show flash:") # For switches
```


Task

```
time.sleep(2)
```

```
p1 = re.compile("\d+(?=\sbytes\sfree\))")
```

```
m1 = p1.findall(showdir)
```

```
flashfree = ((int(m1[0])/1000000))
```

```
print(f"{ip} Free flash size : ", flashfree, "MB")
```

```
print("-"*80)
```

```
if flashfree < (newiossize * 1.5):
```

```
    print(f"Not enough space on {ip}'s flash! Exiting")
```

```
    print("!"*80)
```

```
    exit()
```

```
else:
```

```
    print(f"{ip} has enough space for new IOS.")
```

```
    print("-"*80)
```

```
# Trigger check_flash function to run
```

```
check_flash(device_list_netmiko, newiossize)
```

10 F. This script is from `upload_ios1.py` (`tool7_upload_ios`), and it forms a big part of the main script. As the user uploads the file, some housekeeping needs to take place. First, the logged-in user has a level 15 privilege authorization for this user. If the first and second requirements are not met, you have overlooked your configuration. If the configuration is correctly configured with the right access to this device, then the application checks for `ip scp server enable`. If it is not enabled, it enables it before the new IOS file upload and then disables it after the upload finishes. Note that the next task

```
"""
```

```
Step 6. From upload_ios1.py (tool7_upload_ios) application:
```

```
First, check the aaa configuration requirements. If not met, exit for correct configuration.
```

```
Third, upload the IOS file to the router, then disable ip scp.
```

```
"""
```

```
def ios_upload(device_list_netmiko, device_list):
```

```
# Task
```

```
i = 0
```

```
for device in device_list_netmiko:
```

```
    device_x = device_list[i]
```

```
    newios = device_x[4] # Call out newios value
```

```
    newiosmd5 = device_x[5] # Call out newiosmd5 value
```

```
    i += 1
```

```
    s_newios = f'/home/pynetauto/my_tools/new_ios/{newios}'
```

```
    print(""*80)
```

```
    #print(device)
```

```
    ip = str(device['host'])
```

```
    print(f'{ip} beginning IOS XE upgrade.')
```

```
    username = str(device['username'])
```

```
    net_connect = ConnectHandler(**device)
```

```
    net_connect.send_command("terminal length 0")
```

```
    showrun = net_connect.send_command("show running-config")
```

```
    check_priv15 = (f'username {username} privilege 15')
```

```
    aaa_authentication = "aaa authentication login default local enable"
```

```
    aaa_authorization = "aaa authorization exec default local"
```

```
    if check_priv15 in showrun :
```

```
        print(f'{username} has level 15 privilege - OK')
```

```
    if aaa_authentication in showrun:
```

```
        print("check_aaa_authentication - OK")
```

```
    if aaa_authorization in showrun:
```

```
        print("check_aaa_authorization - OK")
```

```
# Task
```

```
else:
```

```
    print("aaa_authorization - FAILED ")
```

```
    exit()
```

```
else:
```

```
    print("aaa_authentication - FAILED ")
```

```
    exit()
```

```
else:
```

```
    print(f"{username} has not enough privilege - FAILED")
```

```
    exit()
```

```
net_connect.enable(cmd='enable 15')
```

```
net_connect.config_mode()
```

```
net_connect.send_command('ip scp server enable')
```

```
net_connect.exit_config_mode()
```

```
time.sleep(1)
```

```
print("New IOS uploading in progress! Please wait...")
```

```
scp_conn = SCPConn(net_connect)
```

```
scp_conn.scp_transfer_file(s_newios, newios)
```

```
scp_conn.close()
```

```
time.sleep(1)
```

```
net_connect.config_mode()
```

```
net_connect.send_command('no ip scp server enable')
```

```
net_connect.exit_config_mode()
```

- 11 G. In this task, you can borrow the script from `md5_verify2.py` (in `tool8_md5_cisco`) to perform the MD5 verification. Again, we are utilizing the power of regular expressions to test if Cisco IOS XE's `verify` command has run successfully. We are using the `re` (regular expression) compiler and use it as a variable to test the validity of the MD5 value. If any

```
# Task
```

```
"""
```

Step 7. From md5_verify2.py (in tool8_md5_cisco) application :

Fourth, verify the MD5 value of the new IOS on the router flash, then

compare it with the server-side MD5 value.

```
"""
```

```
try:
```

```
    locate_newios = net_connect.send_command(f"show flash: | in {newios}")
```

```
    if newios in locate_newios:
```

```
        result = net_connect.send_command("verify /md5 flash:{} {}".format(newios,newiosmd5))
```

```
        print(result)
```

```
        p1 = re.compile(r'Verified')
```

```
        p2 = re.compile(r'[a-zA-F0-9]{31}[a-zA-F0-9]')
```

```
        verified = p1.findall(result)
```

```
        newiosmd5flash = p2.findall(result)
```

```
        if verified:
```

```
            result = True
```

```
            print("MD5 values MATCH! Continue")
```

```
            print("MD5 of new IOS on Server : ",newiosmd5)
```

```
            print("MD5 of new IOS on flash : ",newiosmd5flash[0])
```

```
            print("-"*80)
```

```
        else:
```

```
            result = False
```

```
            print("MD5 values DO NOT MATCH! Exiting.")
```

```
            print("-"*80)
```

```
# Task
```

```
exit()
```

```
else:
```

```
print("No new IOS found on router's flash. Continue to next device...")
```

```
print("-"*80)
```

```
except (NetMikoTimeoutException):
```

```
print (f'Timeout error to : {ip}')
```

```
print("-"*80)
```

```
continue
```

```
except unknown_error:
```

```
print ('Unknown error occurred : ' + str(unknown_error))
```

```
print("-"*80)
```

```
continue
```

12

H. Under normal circumstances, this may not be required, but to make the application a little more real, I have added a function to reload the router by answering `yes` or `no`. This is a large portion of the script. As soon as you respond with `yes`, the application will reload the running configuration to the startup configuration. This is followed by pre-reload configuration captures of the running configuration. After the reload, you will lose the SSH connection, and also, the Python automation server will return the first 60 seconds; then we run the `check_port_22_post` function to scan port 22 for its status every 5 seconds. After the reload, we run through with the post-reload configuration check; then it will produce multiple HTML comparison files for the configuration.

```
"""
```

```
Step 8. From reload_yesno2.py (in tool9_yes_no) application :
```

```
Fifth, give the user a chance to review the result and let
```

```
him/her decide if he wants to reload the device. This can be removed to automate this process.
Sixth, change the boot system command to boot into new IOS on the
```

```
"""
```

```
yes_list = ['yes', 'y']
```

```
no_list = ['no', 'n']
```

```
print("!"*80)
```

```
# Task
```

```
print("Note: Devices will be reloaded one at a time.")
```

```
resp = input(f"Do you want to reload {ip} now? (y/n)? ").lower()
```

```
if resp in yes_list:
```

```
    print("Reloading devices")
```

```
    #net_connect = ConnectHandler(**device)
```

```
    net_connect.enable(cmd='enable 15')
```

```
    config_commands1 = ['no boot system', 'boot system flash:/' + newios, 'do write memory']
```

```
    output = net_connect.send_config_set(config_commands1)
```

```
    print (output)
```

```
    net_connect.send_command('terminal length 0\n')
```

```
    show_boot = net_connect.send_command('show boot\n')
```

```
    show_dir = net_connect.send_command('dir\n')
```

```
    if newios not in show_dir:
```

```
        print('Unable to locate new IOS on the flash:/. Exiting.')
```

```
        print("-"*80)
```

```
        exit()
```

```
    elif newios not in show_boot:
```

```
        print('Boot system was not correctly configured. Exiting.')
```

```
        print("-"*80)
```

```
        exit()
```

```
    # Seventh, if boot commands are correct, capture another running backup of the device for
```

```
    elif newios in show_boot and newios in show_dir:
```

```
        try:
```

```
            print(f'Found {newios} in show boot')
```

```
# Task
```

```
print("-"*80)
```

```
print(f'{ip} [Pre-reload] Capture configuration')
```

```
net_connect.send_command("terminal length 0")
```

```
time.sleep(1)
```

```
with open(f'{ip}_showver_pre.txt', 'w+') as f1:
```

```
4print("* [Pre-reload] Capturing 'show version'")
```

```
4showver_pre = net_connect.send_command("show version")
```

```
4f1.write(showver_pre)
```

```
time.sleep(1)
```

```
with open(f'{ip}_showrun_pre.txt', 'w+') as f2:
```

```
4print("* [Pre-reload] Capturing 'show running-config'")
```

```
4showrun_pre = net_connect.send_command("show running-config")
```

```
4f2.write(showrun_pre)
```

```
time.sleep(1)
```

```
with open(f'{ip}_showint_pre.txt', 'w+') as f3:
```

```
4print("* [Pre-reload] Capturing 'show ip interface brief'")
```

```
4showint_pre = net_connect.send_command("show ip interface brief")
```

```
4f3.write(showint_pre)
```

```
time.sleep(1)
```

```
with open(f'{ip}_showroute_pre.txt', 'w+') as f4:
```

```
4print("* [Pre-reload] Capturing 'show ip route'")
```

```
4showroute_pre = net_connect.send_command("show ip route")
```

```
4f4.write(showroute_pre)
```

```
time.sleep(1)
```

Task

```
print("-"*80)
```

```
t1 = time.mktime(time.localtime()) # To measure time elapsed for reload
```

```
# Eighth, initiate the device reload
```

```
print(f"{ip} is now reloading.")
```

```
net_connect.send_command('reload', expect_string="[confirm]")
```

```
net_connect.send_command('yes')
```

```
net_connect.send_command('\n')
```

```
print("-"*80)
```

```
# If you remove the "y" or "n" prompt when reloading is initiated, the server will g
```

```
# OSError, use this code to catch the OSError to run the code continuously.
```

```
# If using the interactive as in this example, then it will produce EOFError, so lea
```

```
# code as is
```

```
# except OSError:
```

```
# print("***Device is now reloading. This may take 2-5 minutes.")
```

```
# time.sleep(45)
```

```
# print("-"*80)
```

```
# port = 22
```

```
# check_port_22_post(ip, port, device, t1)
```

```
# print("-"*80)
```

```
except EOFError:
```

```
print("***Device is now reloading. This may take 2-5 minutes.")
```

```
time.sleep(60)
```

```
print("-"*80)
```

```
port = 22
```


Task

```
# Trigger check_port_22_post function for check port 22 and post check
```

```
check_port_22_post(ip, port, device, t1)
```

```
print("-"*80)
```

```
# If chosen 'n', break out of the operation
```

```
elif resp in no_list:
```

```
print("You have chosen to reload the devices later. Exiting the application.")
```

```
break
```

```
print("*"*80)
```

```
# Trigger ios_upload main function
```

```
ios_upload(device_list_netmiko, device_list)
```

13

I. This script needs to go before the main script, which needs to be called within the same file. Place this script within the main script, so Python must be aware of this script. Previously, we briefly discussed how this script port 22 is open or closed with the `delay` (wait) timer of 5 seconds and `retry` value of 90 (times). The IOS-XE upgrade takes three to five minutes. Still, it may take longer if multiple routing protocols are enabled and heavy IP traffic is present.

Once the router interface comes back up and port 22 is detected as open for connection, this application will compare the sets of files, pre and post, are compared using the `diff` library. The library creates four sets of HTML files (tool16_make_backup) application. In this task, two applications are combined into one. If there is another workflow as before. If there are no other devices left for the IOS upgrade, the application will finish and output the results.

```
"""
```

```
Step 9. From post_check2.py (tool10_post_check) and make_backup1.py (tool6_make_backup).
```

```
Check SSH port 22 and then log back in. Then complete the post-upgrade verification check.
for your devices.
```

```
"""
```

```
def check_port_22_post(ip, port, device, t1):
```

```
    retry = 90 # Try 90 times
```

```
    delay = 5 # 5 seconds wait
```

```
    def isOpen(ip, port):
```

```
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```

# Task

s.settimeout(3)

try:

    s.connect((ip, int(port)))

    s.shutdown(socket.SHUT_RDWR)

    return True # The statements after the return statements are not executed.

except:

    return False

finally:

    s.close()

# Retry 90 times until the port is in open state.

for i in range(retry):

    if isOpen(ip, port):

        print(ip + " is back online.")

        t2 = time.mktime(time.localtime()) - t1

        print(f'{ip}'s reload time : {0} seconds".format(t2))

        print(f"Logging back into {ip} to perform post-reload tasks.")

        print("Please wait...")

        # When port 22 is open, start postreload_check here.

        print(ip)

        net_connect = ConnectHandler(**device)

        print(f'*** {ip} is back online.')

        print("-"*80)

        # [Post-reload] Capture configuration

        print(f'{ip} [Post-reload] Capture configuration")

```

```
# Task
```

```
file14 = open(ip + '_showver_post.txt', 'w+')
```

```
net_connect.send_command("terminal length 0")
```

```
showver_postreload = net_connect.send_command("show version")
```

```
print("* [Post-reload] Capturing 'show version'")
```

```
file14.write(showver_postreload)
```

```
file14.close()
```

```
# Successful upgrade will print IP and correct system boot image name.
```

```
p99= re.compile(r'System image file is ".+[\?="\']')
```

```
m99 = p99.findall(showver_postreload)
```

```
print(f"! {ip}'s {m99}.")
```

```
time.sleep(2)
```

```
file15 = open(ip + '_showrun_post.txt', 'w+')
```

```
showrun_postreload = net_connect.send_command("show running-config")
```

```
print("* [Post-reload] Capturing 'show running-config'")
```

```
file15.write(showrun_postreload)
```

```
file15.close()
```

```
time.sleep(2)
```

```
file16 = open(ip + '_showint_post.txt', 'w+')
```

```
showint_postreload = net_connect.send_command("show ip interface brief")
```

```
print("* [Post-reload] Capturing 'show ip interface brief'")
```

```
file16.write(showint_postreload)
```

```
file16.close()
```

```
time.sleep(2)
```

```
file17= open(ip + '_showroute_post.txt', 'w+')
```

```
# Task
```

```
showroute_postreload = net_connect.send_command("show ip route")
```

```
print("* [Post-reload] Capturing 'show ip route'")
```

```
file17.write(showroute_postreload)
```

```
file17.close()
```

```
time.sleep(2)
```

```
# Compare pre and post configurations to create four html files
```

```
showver_pre = "showver_pre"
```

```
showver_post = "showver_post"
```

```
showver_pre_lines = open(ip+ '_showver_pre.txt').readlines() # Convert to strings for
```

```
comparison
```

```
time.sleep(1)
```

```
showver_post_lines = open(ip+ '_showver_post.txt').readlines() #Convert to strings for
```

```
comparison
```

```
time.sleep(1)
```

```
# Note, four arguments required in HtmlDiff function
```

```
difference = difflib.HtmlDiff(wrapcolumn=70).make_file(showver_pre_lines,
```

```
showver_post_lines, showver_pre, showver_post)
```

```
difference_report = open(ip + "_show_ver_comparison.html", "w+")
```

```
difference_report.write(difference) # Write the differences to the difference_report
```

```
difference_report.close()
```

```
time.sleep(1)
```

```
showrun_pre = "showrun_pre"
```

```
showrun_post = "showrun_post"
```

```
showrun_pre_lines = open(ip+ '_showrun_pre.txt').readlines() # Convert to strings for
```

```
# Task
```

```
comparison
```

```
time.sleep(1)
```

```
showrun_post_lines = open(ip+ '_showrun_post.txt').readlines() # Convert to strings f
```

```
comparison
```

```
time.sleep(1)
```

```
difference = difflib.HtmlDiff(wrapcolumn=70).make_file(showrun_pre_lines,
```

```
showrun_post_lines, showrun_pre, showrun_post)
```

```
difference_report = open(ip + "_show_run_comparison.html", "w+")
```

```
difference_report.write(difference) # Write the differences to the difference_report
```

```
difference_report.close()
```

```
time.sleep(1)
```

```
showint_pre = "showint_pre"
```

```
showint_post = "showint_post"
```

```
showint_pre_lines = open(ip+ '_showint_pre.txt').readlines()
```

```
time.sleep(1)
```

```
showint_post_lines = open(ip+ '_showint_post.txt').readlines()
```

```
time.sleep(1)
```

```
difference = difflib.HtmlDiff(wrapcolumn=70).make_file(showint_pre_lines,
```

```
showint_post_lines, showint_pre, showint_post)
```

```
difference_report = open(ip + "_show_int_comparison.html", "w+")
```

```
difference_report.write(difference)
```

```
difference_report.close()
```

```
time.sleep(1)
```

```
showroute_pre = "showroute_pre"
```

Task

`showroute_post = "showroute_post"``showroute_pre_lines = open(ip+ '_showroute_pre.txt').readlines()``time.sleep(1)``showroute_post_lines = open(ip+ '_showroute_post.txt').readlines()``time.sleep(1)``difference = difflib.HtmlDiff(wrapcolumn=70).make_file(showroute_pre_lines,``showroute_post_lines, showroute_pre, showroute_post)``difference_report1 = open(ip + "_show_route_comparison.html", "w+")``difference_report1.write(difference)``difference_report1.close()``time.sleep(1)``print("-"*80)``break``else:``print(f'{ip} is still reloading. Please wait...')``time.sleep(delay)`

- 4 J. Append the following script at the end of task H script. When the final application runs successfully, the script will print the following message. These are the last lines of code in the IOS upgrade application.

`print("Completed new IOS verification.")``print("All tasks completed successfully!")``tt = time.mktime(time.localtime()) - t # Timer finish to measure script running time``print("Total time taken : {0} seconds".format(tt)) # Informational`

- 5 Remember Lego blocks? Like with Lego bricks, we have converted small pieces of scripts and converted them into a single script. For more details, check the GitHub download page for full reference.

URL: https://github.com/pynetauto/apress_pynetauto

Task

Once you have completed building the Cisco IOS XE upgrade application, run it as shown here, and you should see the results in Chapter 18 on `csr1000v-1` and `csr1000v-2` have been deleted from the flash. If the file exists in the flash for greater success with the IOS upgrade testing. We could also write some code to check whether the same file was prepared and uploaded yourself. Hence, leave this script to overwrite any pre-existing files.

K. Now, let's run the final script. Remember, after the IOS uploads to the flash, you will be prompted to select

```
pynetauto@ubuntu20s1:~/my tools/ios_upgrade_tool$ python3 upgrade_crs1000v.py
```

```
Enter Network Admin ID : pynetauto
```

```
Enter Network Admin PWD : *****
```

```
Confirm Network Admin PWD : *****
```

```
Is the secret same as the password? (y/n) : y # Type in 'y' for same secret password
```

```
-----
```

```
192.168.183.111
```

```
PING 192.168.183.111 (192.168.183.111) 56(84) bytes of data.
```

```
64 bytes from 192.168.183.111: icmp_seq=2 ttl=255 time=1.57 ms
```

```
64 bytes from 192.168.183.111: icmp_seq=3 ttl=255 time=1.61 ms
```

```
64 bytes from 192.168.183.111: icmp_seq=4 ttl=255 time=1.62 ms
```

```
--- 192.168.183.111 ping statistics ---
```

```
4 packets transmitted, 3 received, 25% packet loss, time 3019ms
```

```
rtt min/avg/max/mdev = 1.573/1.599/1.617/0.019 ms
```

```
192.168.183.111 22 open
```

```
-----
```

```
192.168.183.222
```

```
PING 192.168.183.222 (192.168.183.222) 56(84) bytes of data.
```

```
64 bytes from 192.168.183.222: icmp_seq=1 ttl=255 time=1.44 ms
```

```
64 bytes from 192.168.183.222: icmp_seq=2 ttl=255 time=1.61 ms
```

```
64 bytes from 192.168.183.222: icmp_seq=3 ttl=255 time=1.58 ms
```

Task

```
64 bytes from 192.168.183.222: icmp_seq=4 ttl=255 time=1.56 ms
```

```
--- 192.168.183.222 ping statistics ---
```

```
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
```

```
rtt min/avg/max/mdev = 1.438/1.545/1.607/0.064 ms
```

```
192.168.183.222 22 open
```

```
-----
```

```
192.168.183.111 csr1000v-1 RT cisco_xe
```

```
77878ae6db8e34de90e2e3e83741bf39
```

```
77878ae6db8e34de90e2e3e83741bf39
```

```
436.57 MB
```

```
MD5 values matched!
```

```
-----
```

```
192.168.183.222 csr1000v-2 RT cisco_xe
```

```
77878ae6db8e34de90e2e3e83741bf39
```

```
77878ae6db8e34de90e2e3e83741bf39
```

```
436.57 MB
```

```
MD5 values matched!
```

```
-----
```

```
192.168.183.111 Free flash size : 6623.248384 MB
```

```
-----
```

```
192.168.183.111 has enough space for a new IOS.
```

```
-----
```

```
192.168.183.222 Free flash size : 6623.55968 MB
```

```
-----
```


Task

```
configure terminal
```

Enter configuration commands, one per line. End with CNTL/Z.

```
csr1000v-1(config)#no boot system
```

```
csr1000v-1(config)#boot system flash:/csr1000v-universalk9.16.09.06.SPA.bin
```

```
csr1000v-1(config)#do write memory
```

Building configuration...

[OK]

```
csr1000v-1(config)#end
```

```
csr1000v-1#
```

Found csr1000v-universalk9.16.09.06.SPA.bin in show boot

192.168.183.111 [Pre-reload] Capture configuration

* [Pre-reload] Capturing 'show version'

* [Pre-reload] Capturing 'show running-config'

* [Pre-reload] Capturing 'show ip interface brief'

* [Pre-reload] Capturing 'show ip route'

192.168.183.111 is now reloading.

***Device is now reloading. This may take 2-5 minutes.

192.168.183.111 is still reloading. Please wait...

192.168.183.111 is still reloading. Please wait...

192.168.183.111 is still reloading. Please wait...

192.168.183.111 is still reloading. Please wait...

Task

192.168.183.111 is still reloading. Please wait...

192.168.183.111 is still reloading. Please wait...

192.168.183.111 is still reloading. Please wait...

192.168.183.111 is still reloading. Please wait...

192.168.183.111 is still reloading. Please wait...

192.168.183.111 is still reloading. Please wait...

192.168.183.111 is back online.

192.168.183.111's reload time : 0 seconds

Logging back into 192.168.183.111 to perform post-reload tasks.

Please wait...

192.168.183.111

*** 192.168.183.111 is back online.

192.168.183.111 [Post-reload] Capture configuration

* [Post-reload] Capturing 'show version'

! 192.168.183.111's ['System image file is "bootflash:/csr1000v-universalk9.16.09.06.SPA.k

* [Post-reload] Capturing 'show running-config'

* [Post-reload] Capturing 'show ip interface brief'

* [Post-reload] Capturing 'show ip route'

192.168.183.222 beginning IOS XE upgrade.

Task

pynetauto has level 15 privilege - OK

check_aaa_authentication - OK

check_aaa_authorization - OK

New IOS uploading in progress! Please wait...

.....

[omitted for brevity]

.....Done!

Verified (bootflash:csr1000v-universalk9.16.09.06.SPA.bin) = 77878ae6db8e34de90e2e3e83741k

MD5 values MATCH! Continue

MD5 of new IOS on Server : 77878ae6db8e34de90e2e3e83741bf39

MD5 of new IOS on flash : 77878ae6db8e34de90e2e3e83741bf39

!!

Note: Devices will be reloaded one at a time.

Do you want to reload 192.168.183.222 now? (y/n)? y # Type in 'y' to reload csr1000v-2

Reloading devices

configure terminal

Enter configuration commands, one per line. End with CNTL/Z.

csr1000v-2(config)#no boot system

csr1000v-2(config)#boot system flash:/csr1000v-universalk9.16.09.06.SPA.bin

csr1000v-2(config)#do write memory

Building configuration...

[OK]

csr1000v-2(config)#end

```
# Task
```

```
csr1000v-2#
```

```
Found csr1000v-universalk9.16.09.06.SPA.bin in show boot
```

```
-----
```

```
192.168.183.222 [Pre-reload] Capture configuration
```

```
* [Pre-reload] Capturing 'show version'
```

```
* [Pre-reload] Capturing 'show running-config'
```

```
* [Pre-reload] Capturing 'show ip interface brief'
```

```
* [Pre-reload] Capturing 'show ip route'
```

```
-----
```

```
192.168.183.222 is now reloading.
```

```
***Device is now reloading. This may take 2-5 minutes.
```

```
-----
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is still reloading. Please wait...
```

```
192.168.183.222 is back online.
```

```
192.168.183.222's reload time : 0 seconds
```

Task

Logging back into 192.168.183.222 to perform post-reload tasks.

Please wait...

192.168.183.222

*** 192.168.183.222 is back online.

192.168.183.222 [Post-reload] Capture configuration

* [Post-reload] Capturing 'show version'

! 192.168.183.222's ['System image file is "bootflash:/csr1000v-universalk9.16.09.06.SPA.k

* [Post-reload] Capturing 'show running-config'

* [Post-reload] Capturing 'show ip interface brief'

* [Post-reload] Capturing 'show ip route'

Completed new IOS verification.

All tasks completed successfully!

Total time taken : 976.0 seconds # = 16.27 minutes in total

After the IOS XE upgrades complete successfully on both `csr1000v-1` and `csr1000v-2`, you will find all files under `/home/pynetauto/my_tools/` directory (see Figure 19-1). Copy and open the files to check the success of your IOS upgrade. You have now upgraded two IOS-XE routers using a handwritten Python network automation code. I hope you have enjoyed this experience.

To watch a demo video of the Cisco IOS-XE upgrade, visit <https://www.youtube.com/watch?v=tnNEeXAGd0M>.

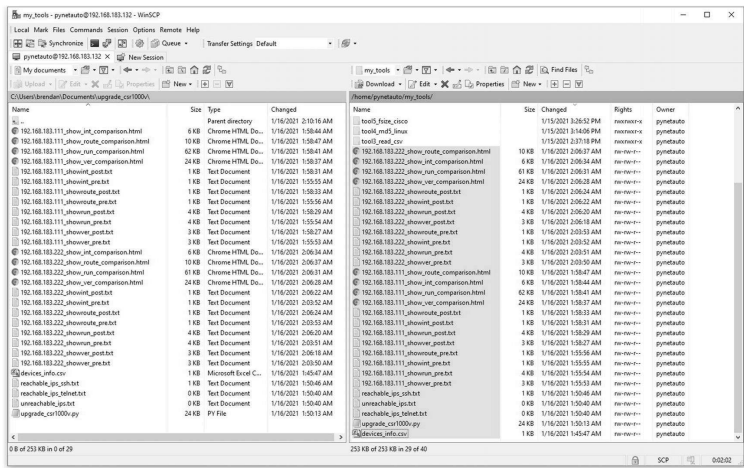


Figure 19-1. WinSCP, copying files to validate successful Cisco router IOS-XE upgrades

Summary

In a real production environment, the scripts are separated into various functioning scripts, called *modules*. Another way to keep the main script lightweight is to save each application as a different module in another file. I have chosen to create this whole application in a single Python script to help you understand how it works, but you probably have already noticed how inefficient it is to work with a lengthy scripted code. Although I can continue writing this code in several separate modules or files and re-import the functions as a module into the main script, I will leave this challenge for you.

I hope your first Python network automation journey was a pleasant surprise and that you are looking forward to the next part of your journey. Together, we have covered a lot of ground in this book, but possibly we have also left many holes open for improvement. You cannot become good at something overnight; writing Python code is not like winning the lottery or becoming an overnight YouTube sensation. It will take many months and years to reach a certain level of confidence in Python programming. Remember Joseph M. Marshall's quote, "Life is a circle. The end of a journey means the start of another one."

Final Words

Taking on a new programming language to take one's career to the next level may feel like a daunting task for many of us. It is like embarking on a new journey into the unknown. Many Technology evangelists encourage people to learn a programming language and start writing code today. Some evangelists preach that Python is for everyone. Here, I disagree politely. Programming or writing code is not for everyone. However, if you spend hours working in front of a computer, learning a high-level programming language like Python can do many wonders at work. If your job involves supporting IT services or infrastructure, I cannot find a single excuse not to learn Python coding. Although Python is one of the easiest programming languages to learn, not many people talk about what happens after reaching a certain level. Python is the same as all other programming languages, and Python or another programming language

is only as bright as the person writing the code. Learning a programming language cannot be a means to an end. Instead, it is another beginning or a new journey with no finish line. So, you always have to keep at school and seek opportunities to apply the concepts and skills you have learned along the journey.

In this book, I tried to write a hands-on Python network automation book for new and existing network engineers, covering various essential IT administration skills required to build an infrastructure from the ground up and write a working application that can help them better manage their network infrastructure. I hope you have enjoyed a broad exposure to various technologies, and you have built a strong foundation towards better enterprise network administration. Lastly, thank you for purchasing my book to start your Python network automation journey. I want to congratulate everyone who has reached this page after running the final lab successfully. Even if you are still working toward completing the final lab, you still have plenty of time to go over the contents of this book as you now own it.

Answers collapsed